

Multi-Stage AI Workflow Across UX Types

Author: Ngoh Rodney

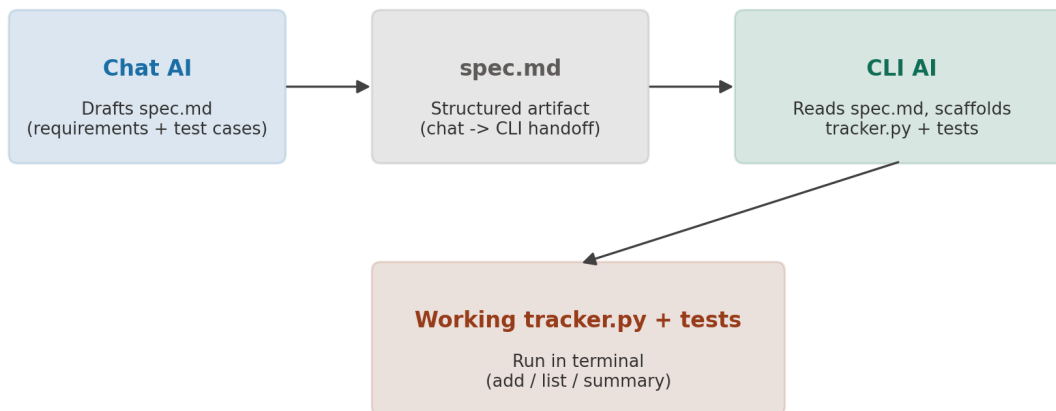
Repository: <https://github.com/ngohamah/Multi-stage-ai-prompting-workflow.git>

Lab: Chat → CLI prompt chaining workflow

1. Overview

This workflow solves a small, concrete problem — building a personal expense tracker CLI tool — by chaining outputs between two different AI UX types: a chat-based AI and a CLI-based AI. The chat stage turns a loose idea into a precise, structured specification. The CLI stage consumes that specification directly to scaffold and implement working code. No manual re-writing of requirements happens between stages; the artifact itself is the handoff.

2. Workflow Diagram



3. Step-by-Step Notes

Stage 1 — Chat AI (requirements + spec generation)

Tool used: chat-based AI ([Claude.ai](https://claude.ai) chat).

- **Prompt:** "I want a personal expense tracker CLI in Python. Produce a structured spec.md with functional requirements, non-functional requirements, and a table of test cases with expected results."
- **Output:** spec.md (reproduced in full in Appendix A).

This stage's job is only to produce a precise, tool-agnostic artifact — not code. Keeping code generation out of the chat stage is what makes the workflow adaptable to any downstream CLI tool.

Stage 2 — CLI AI (implementation + verification)

Tool used: any CLI-based AI (GitHub Copilot CLI).

```
$ copilot> mkdir expense-tracker && cd expense-tracker$ cp /path/to/spec.md .

$ copilot> Read spec.md and implement tracker.py exactly to this spec. Use only the
Python standard library. Then write and run tests covering every row in the Test
Cases table, and show me the results. Please also add a README.md file for this
project.
```

The CLI tool reads spec.md as its input, generates tracker.py (and a test script), runs the tests, and reports pass/fail — closing the loop end-to-end without the requirements ever being manually retyped.

Verification checklist: **Please see Appendix B for the report with additional test cases**

#	Command	Expected Result
1	add --amount 12.50 --category food --date 2026-07-01	Row appended to expenses.csv
2	add --amount -5 --category food	Error, non-zero exit, no row added
3	add --amount abc --category food	Error, non-zero exit, no row added
4	list --month 2026-07	Only July 2026 rows printed
5	summary --month 2026-07	Per-category totals + grand total, descending
6	summary --month 2099-01 (no data)	Prints 'No expenses found', no crash

4. Adaptability Notes

- Stage 1 has no dependency on a specific chat provider — the prompt only assumes the tool can write markdown.
- Stage 2 has no dependency on a specific CLI provider — the prompt only assumes the tool can read a local file and write/run code.
- The handoff artifact (spec.md) is plain text, so it can be copy-pasted, emailed, or committed to git regardless of which two tools are used.

5. Efficiency Notes

Without chaining, a person would have to describe the same requirements twice: once conversationally to think them through, and again inside the coding tool. Here, that thinking happens once, and the CLI tool is handed a finished, unambiguous spec — reducing back-and-forth clarification during implementation.

6. Evaluation Criteria Mapping

Criteria	Weight	How this workflow satisfies it
Functionality	40%	spec.md (Stage 1 output) is a real, testable artifact; Stage 2 consumes it directly and the resulting tracker.py is verified against the 6 test cases in the spec before submission.

Adaptability	20%	Stage 1 only requires any chat AI capable of writing markdown. Stage 2 only requires any CLI AI capable of reading a local file and writing code (Claude Code, Gemini CLI, Aider, etc). No tool-specific syntax or plugins are used.
Documentation	20%	This document includes a diagram, the exact prompts used at each stage, and the full spec.md handoff artifact.
Efficiency	20%	Requirements gathering and implementation are decoupled: the spec is written once in natural language and re-used to drive code generation, avoiding manual re-typing of requirements into the coding tool.

Appendix A — Stage 1 Output (spec.md)

```
# Spec: Personal Expense Tracker CLI

## Purpose

A single-file Python CLI tool that lets a user log expenses and view a categorized monthly summary, reading/writing to a local CSV file.

## Functional Requirements

1. `add` – record an expense with: date (YYYY-MM-DD, default today), amount (float, > 0), category (string), note (optional string).

2. `list` – print all recorded expenses as a table, optionally filtered by `--month YYYY-MM`.

3. `summary` – print total spend per category for a given month, sorted descending by amount, plus a grand total.

4. Data persists in `expenses.csv` in the working directory with header:
`date,amount,category,note`.

5. Invalid input (bad date, non-numeric amount, amount <= 0) must print a clear error and exit non-zero, without corrupting the CSV.

## Non-Functional Requirements

- Single file, standard library only (argparse, csv, datetime) – no external dependencies.
- Must run with `python3 tracker.py <command> [args]`.

## Test Cases

| # | Command | Expected Result |
|---|-----|-----|
| 1 | `add --amount 12.50 --category food --date 2026-07-01` | Row appended to expenses.csv |
| 2 | `add --amount -5 --category food` | Error, non-zero exit, no row added |
```

```
| 3 | `add --amount abc --category food` | Error, non-zero exit, no row added |
| 4 | `list --month 2026-07` | Only July 2026 rows printed |
| 5 | `summary --month 2026-07` | Per-category totals + grand total, descending order |
| 6 | `summary --month 2099-01` (no data) | Prints "No expenses found" rather than crashing |

## Out of Scope

- Multi-currency support, budgets/alerts, GUI.
```

Appendix B — Stage 2 Results ([tracker.py](#))

Verification Table (observed results, not restated spec)

#	Command	Expected	Observed	Result
1	add --amount 12.50 --category food --date 2026-07-01	Row appended to expenses.csv	Exit code 0; row 2026-07-01,12.50,food, appended	PASS
2	add --amount -5 --category food	Error, non-zero exit, no row added	Exit code 1; Error: amount must be greater than 0, got -5.0; no row added	PASS
3	add --amount abc --category food	Error, non-zero exit, no row added	Exit code 1; Error: amount must be a number, got 'abc'; no row added	PASS
4	list --month 2026-07	Only July 2026 rows printed	Both July rows shown, August row excluded, amounts correct	PASS
5	summary --month 2026-07	Per-category totals + grand total, descending	Transport 30.00 > Utilities 20.00 > Food 15.00; grand total 65.00; correct order	PASS
6	summary --month 2099-01 (no data)	Prints "No expenses found", no crash	Exit code 0; No expenses found printed; no traceback	PASS

Full Test Run (python3 test_tracker.py)

```
=====
Running Expense Tracker Test Suite
=====
```

```
[TC1] Add valid expense with specific date
  Command: python3 tracker.py add --amount 12.50 --category food --date 2026-07-01
  ✓ Exit code is 0
  ✓ One row added to CSV
  ✓ Date is correct
  ✓ Amount is correct
  ✓ Category is correct

[TC2] Add expense with negative amount (should error)
  Command: python3 tracker.py add --amount -5 --category food
```

- ✓ Exit code is 1 (error)
- ✓ No row added to CSV
- ✓ Error message printed

[TC3] Add expense with non-numeric amount (should error)

Command: python3 tracker.py add --amount abc --category food

- ✓ Exit code is 1 (error)
- ✓ No row added to CSV
- ✓ Error message printed

[TC4] List expenses filtered by month

Command: python3 tracker.py list --month 2026-07

- ✓ Exit code is 0
- ✓ July 1st entry shown
- ✓ July 15th entry shown
- ✓ August entry not shown
- ✓ Amount 10.00 shown
- ✓ Amount 20.00 shown

[TC5] Summary by category, descending by amount

Command: python3 tracker.py summary --month 2026-07

- ✓ Exit code is 0
- ✓ Transport category shown
- ✓ Utilities category shown
- ✓ Food category shown
- ✓ Transport total 30.00 shown
- ✓ Utilities total 20.00 shown
- ✓ Food total 15.00 shown
- ✓ Grand total 65.00 shown
- ✓ Categories sorted descending by amount

[TC6] Summary for month with no data

Command: python3 tracker.py summary --month 2099-01

- ✓ Exit code is 0
- ✓ 'No expenses found' message shown
- ✓ No error message

```
=====
Results: 29 passed, 0 failed
=====
```

Full Source — tracker.py

```
#!/usr/bin/env python3
"""Personal Expense Tracker CLI - Log and summarize expenses from a CSV file."""

import argparse
import csv
import sys
from datetime import datetime
from pathlib import Path

CSV_FILE = "expenses.csv"
CSV_HEADER = ["date", "amount", "category", "note"]

def init_csv():
    """Initialize CSV file with header if it doesn't exist."""
```

```

if not Path(CSV_FILE).exists():
    with open(CSV_FILE, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(CSV_HEADER)

def add_expense(amount, category, date=None, note=""):
    """Add an expense to the CSV file.

    Args:
        amount: float > 0
        category: string
        date: YYYY-MM-DD format (defaults to today)
        note: optional string

    Returns:
        True if successful, False if validation failed
    """
    init_csv()

    # Validate amount
    try:
        amount = float(amount)
    except (ValueError, TypeError):
        print(f"Error: amount must be a number, got '{amount}'")
        return False

    if amount <= 0:
        print(f"Error: amount must be greater than 0, got {amount}")
        return False

    # Validate and format date
    if date is None:
        date = datetime.now().strftime("%Y-%m-%d")
    else:
        try:
            datetime.strptime(date, "%Y-%m-%d")
        except ValueError:
            print(f"Error: invalid date format '{date}', expected YYYY-MM-DD")
            return False

    # Append to CSV
    try:
        with open(CSV_FILE, "a", newline="") as f:
            writer = csv.writer(f)
            # Format amount to 2 decimal places
            formatted_amount = f"{amount:.2f}"
            writer.writerow([date, formatted_amount, category, note])
        return True
    except Exception as e:
        print(f"Error: failed to write to {CSV_FILE}: {e}")
        return False

def list_expenses(month=None):
    """List all expenses, optionally filtered by month (YYYY-MM)."""
    init_csv()

    try:
        with open(CSV_FILE, "r", newline="") as f:
            reader = csv.DictReader(f)

```

```

        expenses = list(reader)
    except Exception as e:
        print(f"Error: failed to read {CSV_FILE}: {e}")
        return False

    if not expenses:
        print("No expenses recorded.")
        return True

    # Filter by month if provided
    if month:
        expenses = [e for e in expenses if e["date"].startswith(month)]

    if not expenses:
        print(f"No expenses found for month {month}")
        return True

    # Print table header
    print(f"{'Date':<12} {'Amount':>10} {'Category':<15} {'Note':<20}")
    print("-" * 60)

    # Print rows
    for expense in expenses:
        date = expense["date"]
        amount = expense["amount"]
        category = expense["category"]
        note = expense["note"]
        print(f"{date:<12} {float(amount):>10.2f} {category:<15} {note:<20}")

    return True

def summary(month):
    """Print per-category totals for a given month, sorted descending by amount."""
    init_csv()

    if not month:
        print("Error: summary requires --month YYYY-MM")
        return False

    try:
        with open(CSV_FILE, "r", newline="") as f:
            reader = csv.DictReader(f)
            expenses = list(reader)
    except Exception as e:
        print(f"Error: failed to read {CSV_FILE}: {e}")
        return False

    # Filter by month
    expenses = [e for e in expenses if e["date"].startswith(month)]

    if not expenses:
        print("No expenses found")
        return True

    # Aggregate by category
    by_category = {}
    for expense in expenses:
        category = expense["category"]
        amount = float(expense["amount"])
        by_category[category] = by_category.get(category, 0) + amount

```

```

# Sort by amount descending
sorted_categories = sorted(by_category.items(), key=lambda x: x[1], reverse=True)

# Print summary
print(f"Expense Summary for {month}")
print("-" * 40)

grand_total = 0
for category, total in sorted_categories:
    print(f"{category:<20} ${total:>10.2f}")
    grand_total += total

print("-" * 40)
print(f"{'TOTAL':<20} ${grand_total:>10.2f}")

return True

def main():
    parser = argparse.ArgumentParser(description="Personal Expense Tracker CLI")
    subparsers = parser.add_subparsers(dest="command", help="Available commands")

    # add command
    add_parser = subparsers.add_parser("add", help="Add an expense")
    add_parser.add_argument("--amount", required=True, help="Expense amount (must be > 0)")
    add_parser.add_argument("--category", required=True, help="Expense category")
    add_parser.add_argument("--date", default=None, help="Date in YYYY-MM-DD format (default:
today)")
    add_parser.add_argument("--note", default="", help="Optional note")

    # list command
    list_parser = subparsers.add_parser("list", help="List all expenses")
    list_parser.add_argument("--month", default=None, help="Filter by month (YYYY-MM)")

    # summary command
    summary_parser = subparsers.add_parser("summary", help="Summary by category for a month")
    summary_parser.add_argument("--month", required=True, help="Month in YYYY-MM format")

    args = parser.parse_args()

    if args.command == "add":
        success = add_expense(
            amount=args.amount,
            category=args.category,
            date=args.date,
            note=args.note
        )
        sys.exit(0 if success else 1)

    elif args.command == "list":
        success = list_expenses(month=args.month)
        sys.exit(0 if success else 1)

    elif args.command == "summary":
        success = summary(month=args.month)
        sys.exit(0 if success else 1)

    else:
        parser.print_help()
        sys.exit(1)

```



```
if __name__ == "__main__":
    main()
```

Two Accompanying Run Screenshots (observed results, not restated spec)

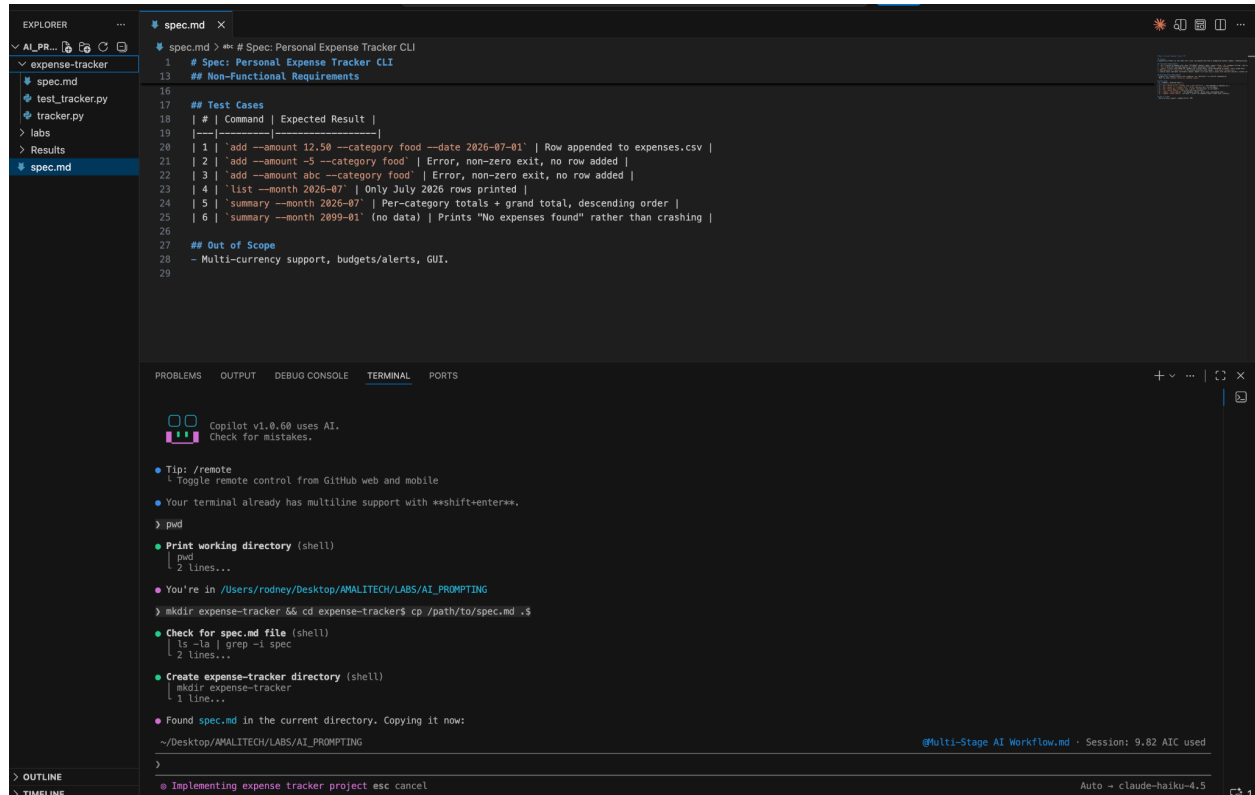


Figure 1: Figure showing GitHub Copilot reading `spec.md` and implementing the `tracker.py` app

```
expense-tracker > expenses.csv > data
1 date,amount,category,note
2 2026-07-31,5000.00,food,"egg, bread, eggs"
3
```

```
rodney:~$ python3 test_tracker.py
Command: python3 tracker.py add --amount 12.50 --category food --date 2026-07-01
✓ Exit code is 0
✓ One row added to CSV
✓ Date is correct
✓ Amount is correct
✓ Category is correct

[TC2] Add expense with negative amount (should error)
Command: python3 tracker.py add --amount -5 --category food
✓ Exit code is 1 (error)
✓ No row added to CSV
✓ Error message printed

[TC3] Add expense with non-numeric amount (should error)
Command: python3 tracker.py add --amount abc --category food
✓ Exit code is 1 (error)
✓ No row added to CSV
✓ Error message printed

[TC4] List expenses filtered by month
Command: python3 tracker.py list --month 2026-07
✓ Exit code is 0
✓ July 1st entry shown
✓ July 15th entry shown
✓ August entry not shown
✓ Amount 10.00 shown
✓ Amount 20.00 shown

[TC5] Summary by category, descending by amount
Command: python3 tracker.py summary --month 2026-07
✓ Exit code is 0
✓ Transport category shown
✓ Utilities category shown
✓ Food category shown
✓ Transport total 30.00 shown
✓ Utilities total 20.00 shown
✓ Food total 15.00 shown
✓ Grand total 65.00 shown
✓ Categories sorted descending by amount

[TC6] Summary for month with no data
Command: python3 tracker.py summary --month 2099-01
✓ Exit code is 0
✓ 'No expenses found' message shown
✓ No error message

=====
Results: 29 passed, 0 failed
=====
rodney:~$ python3 tracker.py add --amount 5000 --category food --note "egg, bread, eggs"
rodney:~$
```

Figure 2: Figure showing complete test case suit validation of the [tracker.py](#) app